

Lab Practicum 01: Empirical Parallel Computing

Silicon Profiling, Multi-Core Scaling Limits, and Concurrency Hazards

Duration: 90 Minutes

Format: Solo In-Class Lab

Weight: 5% of Course Grade

STUDENT NAME	STUDENT ID	HOST CPU / SOC MODEL	OPERATING SYSTEM & KERNEL
Rakhat Bitimbay	230107142	Apple M2	MacOS 26.3.1 Darwin Kernel 25.6.0

Generative AI & Tooling Policy: You are explicitly authorized to use LLMs (ChatGPT, Claude, GitHub Copilot) to generate code scripts in any language (C, C++, Python, Go, Rust, Java). However: All execution timings, memory counters, and hardware cache metrics must be physically benchmarked on your host machine. Fabricated numbers receive zero credit.

TASK 1: Host Silicon Audit & Flynn's Taxonomy Mapping

15 Minutes | 20 Points

Query your operating system and CPU registers to document physical core topology, cache boundaries, and vector math extensions.

Terminal Audit Commands: Linux: lscpu && lscpu --caches | macOS: sysctl -a | grep machdep.cpu | Windows: Run wmic cpu get /format:list and inspect Task Manager > Performance > CPU.

Hardware Property	Local Measured Value	Hardware Property	Local Measured Value
CPU Model & Architecture	Apple M2 (ARM64)	Base & Max Boost Clock	Not directly exposed by macOS
Physical Hardware Cores	8	Logical Cores (SMT/Threads)	8
L1 Data / Instruction Cache	P-cores: 128 KB D / 192 KB E-cores: 64 KB D / 128 KB	L2 Cache (Per Core / Total)	P: 16 MB; E: 4 MB
L3 Shared Last-Level Cache	Not exposed	SIMD Vector Extensions	ARM NEON

Q1.1: Map your host machine's execution modes into Flynn's Taxonomy:

My Apple M2 operates in different modes depending on the workload. SISD occurs when one CPU core executes a single instruction stream on individual data, such as a single-threaded program. SIMD is supported through ARM NEON/Advanced SIMD, allowing one instruction to operate on multiple data elements at the same time, which is useful for vector and matrix calculations. MIMD occurs when multiple CPU cores execute different instruction streams simultaneously, for example when macOS runs multiple threads or processes across the Performance and Efficiency cores. Therefore, the M2 can support SISD, SIMD, and MIMD execution depending on the application and workload.

TASK 2: Multi-Thread Scaling Benchmark & Contention Wall

35 Minutes | 30 Points

Write or prompt an AI to generate a compute-bound CPU benchmark parameterized by thread count N. Run 3 iterations per thread count and log wall-clock execution time.

Workload Algorithm: [e.g., Prime Sieve to 5,000,000 / Matrix Multiply]				Language & Runtime: [e.g., C OpenMP / Python multiprocessing / Go]		
Threads (N)	Run 1 (s)	Run 2 (s)	Run 3 (s)	Avg Time TN (s)	Speedup SN = T1/TN	Efficiency EN = SN/N
N = 1 (Baseline)					1.00×	100.0%
N = 2						
N = 4						
N = 8						
N = 16						
N = 32						

Q2.1: Scaling Saturation & Hardware Contention Analysis:

Identify the exact thread count where speedup sub-linearly saturated or reversed. Analyze this inflection point by citing your hardware's physical vs. logical core ratio, OS thread scheduling overhead, cache thrashing, or thermal throttling:

TASK 3: The Unsynchronized Shared Counter & Race Condition Trap

25 Minutes | 25 Points

Spawn 10 parallel threads. Each thread executes a loop incrementing a shared global integer 1,000,000 times without atomic operations or locks. Theoretical expected total: $10 \times 1,000,000 = 10,000,000$. Execute 10 consecutive runs.

Run	Measured Output	Error (10^7 - Act)	Run	Measured Output	Error (10^7 - Act)
#1			#6		
#2			#7		
#3			#8		
#4			#9		
#5			#10		

Q3.1: Read-Modify-Write (RMW) Window Deconstruct why value corruption occurs at the machine instruction level (Load → Add → Store) when two cores interleave execution on shared memory:	Q3.2: The Synchronization Penalty Wrap the counter in a Mutex/Lock. Record execution time: Unlocked = _____ ms vs. Locked = _____ ms. Why is race-free synchronized code substantially slower?
--	--

TASK 4: Empirical Amdahl Verification vs. Gustafson Scaling Horizon

15 Minutes | 15 Points

Use your empirical single-thread time (T1) and two-thread time (T2) from Task 2 to derive the parallel fraction p of your workload, then evaluate theoretical performance bounds.

1. Parallel Fraction Derivation (p) Formula: $p = 2 \times (T1 - T2) / T1$ T1 = _____ s T2 = _____ s Calculated p = _____ (decimal between 0.0 and 1.0)	2. Amdahl Theoretical Speedup Ceiling Formula: $S_{max} = 1 / (1 - p)$ Sequential fraction $(1 - p) =$ _____ Max Theoretical Speedup ($N \rightarrow \infty$) = _____ ×
3. Amdahl 64-Core Projection Formula: $S_{64} = 1 / [(1 - p) + (p / 64)]$ Projected Speedup on 64 Cores = _____ ×	4. Gustafson Scaled Speedup (Weak Scaling) Formula: $SGustafson = (1 - p) + (p \times 64)$ Projected Scaled Speedup (64× Problem) = _____ ×

Q4.1: Strong vs. Weak Scaling Analysis:

Why does Gustafson's model predict near-linear scaling on a 64-core node, whereas Amdahl's formula predicts a hard bottleneck for the exact same algorithm?

- Submission Guidelines & Deliverables Checklist:**
- Complete all 4 tables and technical reflection prompts directly on this document (or submit as an annotated Word / PDF).
 - Attach a single terminal screenshot verifying Task 2 executing across active threads (e.g., htop, top, or Task Manager CPU pane).
 - Upload the final .docx / PDF to the course portal before the conclusion of the 90-minute lab period.